LAS CIEGAS HORMIGAS ELÉCTRICAS

Ingeniería Aumentada con Sistemas Inteligentes

Volumen II

De la experiencia al Método

Javier G. Grandez

v1.0 -2026

Tabla de contenidos

.1. Laboratorios 5
Laboratorios 7
I. AI Labs 9
I.1. Prompts, contexto y sesiones 13
I.2. Paso 1 — Repite 15
I.3. Paso 2 — La misma necesidad, distintos prompts 17
I.4. Paso 3 — Saca información fuera del prompt 19
I.5. Paso 4 — Cambia de sesión 21
I.6. Paso 5 — Observa 23
I.7. Contexto 25
I.8. Qué vamos a hacer 35
I.9. Paso 1 — Construye contexto en una conversación 37
I.10. Paso 2 — Cambia de sesión 39
I.11. Paso 3 — Saca el contexto de la conversación 41
I.12. Paso 4 — Delimita el contexto 43
I.13. Paso 5 — Modifica el contexto 45
I.14. Paso 6 — Crea un conflicto 47
I.15. Paso 7 — Conclusiones 49
I.16. Instrucciones 51
I.17. Paso 1 — Contexto sin reglas de trabajo 53
I.18. Paso 2 — Añade instrucciones al prompt 55
I.19. Paso 3 — Saca las instrucciones del prompt 57
I.20. Paso 4 — Reutiliza las instrucciones 59
I.21. Paso 5 — Modifica las instrucciones 61
I.22. Paso 6 — Crea un conflicto entre instrucciones 63
I.23. Paso 7 — Conclusiones 65
II. Agentic Systems Labs 67
III. Agentic Systems Labs 69
III.1. Agentic Systems 73
IV. Exploratory Labs 75
V. Exploratory Labs 77
V.1. Exploratory Labs 81
VI. IASI Labs 83
VI.1. Infraestructura 87
VI.2. Aislamiento 89
VI.3. Máquina virtual 95

VI.4. Sistema operativo base **97**
VI.5. Documentación **101**
VI.6. Desarrollo **105**
VI.7. Docker y servicios **109**
VI.8. Sistemas inteligentes **117**
VI.9. IASI **121**
VI.10. Productos IASI **123**
VI.11. Postinstalación **125**
Licencias **127**

.1. Laboratorios

Laboratorios

Este volumen reúne laboratorios prácticos organizados por áreas de experimentación. No es necesario recorrerlo de forma lineal: cada parte constituye una puerta de entrada distinta y puede abordarse según la experiencia previa del lector.

.1.1. Part I · AI Labs

Laboratorios sobre interacción con Sistemas Inteligentes, prompts, contexto, instrucciones e infraestructura básica.

Si estos fundamentos ya forman parte de tu práctica habitual, puedes pasar directamente a la Parte II.

Ir a Part I · AI Labs

.1.2. Part II · Agentic Systems Labs

Laboratorios orientados a Sistemas Inteligentes capaces de utilizar herramientas y participar activamente en flujos de trabajo: MCP, commands, tools, skills y mecanismos relacionados.

Si ya trabajas con estos componentes y sus patrones de integración, puedes continuar en las partes siguientes.

Ir a Part II · Agentic Systems Labs

.1.3. Part III · Exploratory Labs

Laboratorios experimentales que exploran tecnologías, ideas o combinaciones que no necesitan formar parte de un recorrido único.

Ir a Part III · Exploratory Labs

.1.4. Part IV · IASI Labs

Laboratorios propios de IASI: experimentos sobre herramientas, procesos, artefactos y prácticas de Ingeniería Aumentada desarrollados dentro del propio ecosistema IASI.

Ir a Part IV · IASI Labs

Parte I.

AI Labs

Esta parte reúne los laboratorios de base sobre el trabajo con Sistemas Inteligentes: prompts, contexto, instrucciones y preparación del entorno de experimentación.

Su objetivo es proporcionar una base práctica común. Quien ya domine estos fundamentos puede utilizar esta parte como referencia o pasar directamente a **Agentic Systems Labs**.

I.1. Prompts, contexto y sesiones

En este primer laboratorio no vamos a estudiar cómo funciona internamente un Sistema Inteligente. Vamos a utilizarlo.

La idea es hacer pruebas muy sencillas, observar qué ocurre y empezar a desarrollar intuición sobre cómo se comporta un Sistema Inteligente cuando cambian las condiciones de una interacción.

No buscamos demostrar que tenga que aparecer una respuesta concreta. De hecho, en algunas pruebas puede que no observes ninguna diferencia apreciable.

Eso también es un resultado.

Para reducir el contexto previo, utilizaremos navegaciones privadas o de incógnito y, cuando queramos empezar desde cero, cerraremos completamente la sesión privada antes de abrir otra.

I.1.1. Qué vamos a hacer

Durante el laboratorio vamos a:

- repetir exactamente la misma petición en dos sesiones separadas;
- expresar una misma necesidad mediante prompts diferentes;
- sacar parte de la información fuera del prompt y colocarla en el contexto de la conversación;
- cambiar de sesión y observar qué ocurre con ese contexto;
- realizar nuestras propias variaciones sobre los experimentos.

No hace falta intentar explicar todavía por qué sucede cada cosa.

Primero vamos a probar.

I.1.2. Qué necesitas

Solo necesitas:

- un navegador con modo privado o de incógnito;
- acceso a un Sistema Inteligente con interfaz conversacional;
- poder cerrar completamente una sesión privada y abrir otra nueva.

No necesitas instalar nada ni escribir código.

I.1.3. Qué aprenderás

Al terminar el laboratorio habrás experimentado que una interacción con un Sistema Inteligente no depende únicamente de la última frase que escribes.

También habrás empezado a distinguir entre la necesidad que quieres resolver, el prompt con el que la expresas, el contexto acumulado durante una conversación y la sesión en la que todo ello ocurre.

Pero el objetivo principal no es memorizar esos conceptos.

Es empezar a experimentar.

I.2. Paso 1 — Repite

Vamos a empezar sin cambiar nada.

Abre una **nueva navegación privada** y escribe:

> Explica qué es una base de datos relacional.

Lee la respuesta y consérvala para poder compararla después.

Ahora cierra completamente esa navegación privada.

Abre una **nueva navegación privada** y escribe exactamente lo mismo:

> Explica qué es una base de datos relacional.

I.2.0.0.1. Observa

Compara ambas respuestas.

Pueden ser diferentes.

Pueden ser muy parecidas.

Pueden incluso ser idénticas.

No esperamos un resultado concreto.

Fíjate simplemente en lo que ha ocurrido.

¿Mantienen la misma estructura?

¿Utilizan los mismos ejemplos?

¿Explican exactamente los mismos conceptos?

¿Cambian el nivel de detalle o la forma de expresarlos?

Si las dos respuestas son idénticas, no intentes forzar una diferencia. Anótalo mentalmente y continúa.

Ese resultado también forma parte del experimento.

I.2.0.0.2. Experimenta

Antes de pasar al siguiente paso puedes repetir la prueba con otra pregunta.

Elige algo que te interese.

Haz la misma petición en dos sesiones privadas distintas.

Prueba dos veces, tres, cinco.

No busques conseguir un resultado determinado.

Mira qué ocurre.

I.3. Paso 2 — La misma necesidad, distintos prompts

Ahora vamos a cambiar deliberadamente una cosa.

Seguiremos intentando resolver la misma necesidad:

> Entender qué es una base de datos relacional.

Abre una nueva navegación privada y escribe:

> Explica qué es una base de datos relacional.

Guarda la respuesta.

Cierra completamente esa navegación privada y abre otra.

Ahora escribe:

> Explica qué es una base de datos relacional usando una biblioteca como analogía.

I.3.0.0.1. Observa

La necesidad de fondo es la misma.

Lo que hemos cambiado es la forma de expresarla.

Compara las respuestas.

¿Explican el mismo concepto?

¿Qué cambia en la estructura?

¿Qué ejemplos aparecen?

¿Qué parte de la segunda respuesta parece consecuencia directa de la instrucción añadida?

No estamos intentando averiguar qué prompt es mejor.

Estamos observando cómo una misma necesidad puede producir respuestas distintas cuando cambiamos la información que incluimos en el prompt.

I.3.0.0.2. Experimenta

Ahora hazlo a tu manera.

Mantén la misma pregunta y cambia alguna condición del prompt.

Por ejemplo, puedes pedir:

- una explicación muy breve;
- una explicación para una persona sin conocimientos técnicos;
- una explicación basada en una analogía diferente;
- una explicación con un ejemplo;
- una comparación con otro concepto.

O inventa tú otra variante.

Haz varios intentos.

Observa qué cambia y qué permanece.

I.4. Paso 3 — Saca información fuera del prompt

Hasta ahora hemos colocado toda la información dentro del propio prompt.

Vamos a cambiar eso.

Cierra la navegación privada anterior y abre una nueva.

Primero escribe:

> Durante esta conversación, cuando expliques conceptos técnicos utiliza una biblioteca como analogía siempre que resulte posible.

Espera la respuesta del sistema.

Después escribe únicamente:

> Explica qué es una base de datos relacional.

I.4.0.0.1. Observa

Compara esta respuesta con la obtenida en el paso anterior cuando escribiste:

> Explica qué es una base de datos relacional usando una biblioteca como analogía.

La necesidad es la misma.

La instrucción sobre la biblioteca también existe en ambos casos.

Pero ahora esa instrucción **no aparece en el último prompt**.

Ha aparecido antes, dentro de la conversación.

¿La respuesta sigue utilizando la analogía?

¿Se parece a la obtenida anteriormente?

¿Hasta qué punto parece influir algo que ya no está escrito en la petición actual?

No necesitamos que las respuestas sean idénticas.

Nos interesa comprobar que el último mensaje no es necesariamente toda la información que participa en la interacción.

I.4.0.0.2. Experimenta

Cambia la instrucción inicial.

Por ejemplo:

> Durante esta conversación, responde siempre con ejemplos relacionados con cocina.

Después haz varias preguntas breves.

O establece otra regla que se te ocurra.

Comprueba cuánto tiempo parece mantenerse.

Cámbiala a mitad de la conversación.

Contradícela.

Haz una excepción.

No hay un resultado que tengas que conseguir.

Prueba y observa.

I.5. Paso 4 — Cambia de sesión

Mantén abierta la navegación privada del paso anterior.

Vuelve a escribir:

> Explica qué es una base de datos relacional.

Observa si el sistema continúa aplicando la instrucción que habías dado antes.

Ahora cierra **completamente** esa navegación privada.

Abre una nueva navegación privada.

No vuelvas a proporcionar la instrucción anterior.

Escribe únicamente:

> Explica qué es una base de datos relacional.

I.5.0.0.1. Observa

Compara esta respuesta con la anterior.

¿Qué información estaba disponible en la sesión anterior?

¿Qué ocurre ahora?

¿Sigue apareciendo la instrucción que habías establecido?

La prueba no pretende demostrar que todos los Sistemas Inteligentes o todas las plataformas gestionen las sesiones de la misma manera.

Queremos comprobar algo más práctico:

no debemos dar por supuesto que el contexto creado en una sesión estará disponible automáticamente en otra.

I.5.0.0.2. Experimenta

Haz tus propias pruebas.

Introduce una palabra inventada y dale un significado dentro de una conversación.

Define una regla.

Establece una preferencia.

Pide al sistema que recuerde algo durante esa sesión.

Después abre otra navegación privada y comprueba qué ocurre.

Prueba también a cerrar el navegador y volver a una conversación existente, si la plataforma lo permite.

Compara situaciones.

Hazte preguntas.

I.6. Paso 5 — Observa

Durante el laboratorio hemos partido de necesidades muy sencillas, pero hemos cambiado varias cosas alrededor de ellas.

Primero repetimos exactamente la misma petición sin esperar que tuviera que ocurrir nada concreto.

Después mantuvimos la misma necesidad y cambiamos el prompt.

Más tarde sacamos parte de la información fuera del prompt y la introdujimos previamente en la conversación.

Finalmente cambiamos de sesión.

Ahora vuelve sobre lo que has observado.

I.6.0.0.1. Algunas preguntas

No busques una respuesta académica. Intenta responder con tus propias palabras.

- ¿Es lo mismo una pregunta que un prompt?
- ¿Dos prompts idénticos tienen que producir necesariamente respuestas distintas?
- ¿Dos prompts diferentes pueden intentar resolver exactamente la misma necesidad?
- ¿Puede una respuesta depender de información que no aparece en el último mensaje?
- ¿Qué ocurre con esa información cuando cambia la sesión?
- ¿Qué diferencias has encontrado entre tus propias pruebas?
- ¿Qué prueba te ha sorprendido más?
- ¿Qué otra cosa te gustaría comprobar?

I.6.0.0.2. Sigue experimentando

El laboratorio termina aquí, pero no tienes por qué detener la prueba.

Cambia las preguntas.

Cambia el orden.

Repite un experimento varias veces.

Introduce instrucciones absurdas o muy específicas.

Prueba con otra plataforma.

Compara dos Sistemas Inteligentes.

Vuelve mañana y repite alguna prueba.

La intención de estos Labs no es darte una secuencia de instrucciones para que obtengas una respuesta que nosotros ya conocemos.

Es ofrecerte un entorno sencillo en el que puedas **probar, observar y empezar a construir tu propia intuición** sobre cómo se comportan los Sistemas Inteligentes.

Más adelante pondremos nombre y estructura a muchas de las cosas que estamos observando.

Por ahora, experimenta.

I.7. Contexto

En el laboratorio anterior observamos que una interacción con un Sistema Inteligente no depende únicamente del último mensaje.

La conversación va acumulando información y esa información puede influir en las respuestas posteriores.

Ahora vamos a experimentar con ese fenómeno de forma deliberada.

Trabajaremos con un proyecto ficticio llamado **Faro** y veremos qué ocurre cuando el contexto vive dentro de una conversación, cuando lo sacamos a un fichero y cuando cambiamos ese fichero.

I.7.1. Qué vamos a hacer

Vamos a:

- construir contexto dentro de una conversación;
- comprobar qué ocurre al cambiar de sesión;
- externalizar ese contexto en un fichero;
- utilizarlo en una sesión nueva;
- preguntar por información definida y no definida;
- modificar el contexto;
- crear conflictos entre distintas fuentes;
- experimentar con nuestras propias variantes.

No buscamos obtener respuestas predeterminadas.

Buscamos observar cómo cambia el comportamiento del Sistema Inteligente cuando cambia el contexto disponible.

I.7.2. Qué necesitas

Necesitas:

- un navegador con modo privado o de incógnito;
- acceso a un Sistema Inteligente con interfaz conversacional;
- poder proporcionar un fichero Markdown a la conversación;
- un editor de texto para modificar `context.md`.

El laboratorio incluye un fichero `context.md` inicial.

I.7.3. Qué aprenderás

Al terminar habrás experimentado que el contexto puede existir dentro de una conversación, pero también puede convertirse en un artefacto explícito que podemos conservar, revisar, modificar, reutilizar y controlar.

Primero vamos a comprobarlo.

I.7.4. Paso 1 — Construye contexto en una conversación

Abre una nueva navegación privada.

Escribe:

> Estamos trabajando en un proyecto llamado Faro.
>
> Faro es una aplicación para gestionar bibliotecas municipales.
>
> Los usuarios pueden buscar libros, reservarlos y consultar sus préstamos.
>
> Faro no vende libros.
>
> Cuando hablamos de "usuario" nos referimos al lector de la biblioteca.

Después pregunta:

> Propón tres funcionalidades nuevas para Faro.

Ahora pregunta:

> ¿Faro vende libros?

Y después:

> ¿Quién es el usuario de Faro?

I.7.4.1. Observa

Las últimas preguntas son muy breves.

Sin embargo, el Sistema Inteligente dispone de información que apareció antes en la conversación.

La sesión ha ido acumulando contexto.

Ahora introduce algo nuevo:

> A partir de ahora, cuando hablemos de una sede nos referiremos a una biblioteca municipal concreta.

Después pregunta:

> ¿Qué significa "sede" en Faro?

Comprueba si el sistema utiliza la definición que acabas de introducir.

I.7.4.2. Experimenta

Añade nuevos conceptos.

Corrige alguno.

Introduce una decisión.

Haz una pregunta que dependa de algo dicho varios mensajes antes.

Observa cómo, a medida que avanza la conversación, el sistema parece ir **aprendiendo a trabajar dentro del contexto que estáis construyendo juntos**.

I.7.5. Paso 2 — Cambia de sesión

Antes de cerrar la sesión anterior, pregunta:

> ¿Qué significa "sede" en Faro?

Comprueba la respuesta.

Ahora cierra completamente la navegación privada.

Abre otra nueva.

Sin proporcionar ninguna información sobre Faro, pregunta:

> ¿Qué significa "sede" en Faro?

Después pregunta:

> ¿Faro vende libros?

I.7.5.1. Observa

Compara las respuestas con las de la sesión anterior.

No esperamos un comportamiento idéntico en todos los Sistemas Inteligentes o plataformas.

Puede existir memoria persistente, información de proyecto, preferencias u otros mecanismos.

Lo importante es comprobar algo práctico:

no podemos asumir que todo lo aprendido durante una sesión seguirá disponible en una sesión nueva.

Parte puede mantenerse.

Parte puede desaparecer.

Parte puede reaparecer por mecanismos distintos al historial de conversación.

I.7.5.2. Experimenta

Introduce en una sesión una palabra inventada.

Dale un significado.

Úsala varias veces.

Después abre una nueva sesión privada y prueba de nuevo.

Haz lo mismo con una decisión, una restricción o una preferencia.

Observa qué se conserva y qué no.

I.7.6. Paso 3 — Saca el contexto de la conversación

Ahora vamos a dejar de depender exclusivamente de lo que la sesión conserve.

Abre `context.md`.

El fichero contiene una descripción de Faro.

Cierra cualquier navegación privada anterior y abre una nueva.

Proporciona `context.md` al Sistema Inteligente y escribe:

> Para este ejercicio, utiliza `context.md` como contexto del proyecto Faro.
>
> Si una respuesta depende de información que no aparece definida en ese fichero, indícalo en lugar de inventarla.

Después pregunta:

> ¿Faro vende libros?
>
> ¿Quién es el usuario de Faro?
>
> ¿Qué significa "sede" en Faro?

I.7.6.1. Observa

La sesión es nueva.

No hemos reconstruido Faro mediante una conversación larga.

Hemos proporcionado explícitamente una parte del conocimiento necesario.

Ahora ese conocimiento no depende únicamente de que una sesión anterior lo recuerde.

Podemos llevarlo con nosotros.

I.7.7. Paso 4 — Delimita el contexto

Seguimos utilizando `context.md`.

Pregunta:

> ¿Faro permite reservar salas de estudio?

Después:

> ¿Qué sistema de base de datos utiliza Faro?

Y finalmente:

> ¿Cuántas bibliotecas utilizan actualmente Faro?

Busca esas respuestas en `context.md`.

No están definidas.

I.7.7.1. Observa

El contexto no solo aporta información.

También puede establecer una frontera entre lo que hemos definido y lo que todavía no hemos definido.

Comprueba qué hace el Sistema Inteligente cuando llega a esa frontera.

¿Reconoce que falta información?

¿Hace una inferencia?

¿Propone una posibilidad?

¿Presenta como cierto algo que el fichero no dice?

No des por supuesto que el comportamiento será perfecto.

Eso también forma parte del experimento.

I.7.7.2. Experimenta

Haz preguntas cuya respuesta esté claramente en el fichero.

Después haz preguntas que solo puedan responderse mediante inferencia.

Prueba con algo ambiguo.

Prueba con algo completamente ausente.

Observa la diferencia entre lo definido, lo inferido, lo propuesto y lo desconocido.

I.7.8. Paso 5 — Modifica el contexto

Abre `context.md` y añade:

> Faro permite reservar salas de estudio.

Guarda el fichero.

Cierra la navegación privada actual y abre otra nueva.

Proporciona la nueva versión de `context.md` y vuelve a indicar:

> Utiliza `context.md` como contexto del proyecto Faro.
>
> Si una respuesta depende de información no definida en el fichero, indícalo.

Pregunta:

> ¿Faro permite reservar salas de estudio?

I.7.8.1. Observa

La pregunta es la misma que antes.

Lo que ha cambiado es el contexto.

Ahora cambia otra cosa.

Sustituye:

> Faro no vende libros.

por:

> Faro permite comprar libros retirados del catálogo de la biblioteca.

Repite la prueba en una sesión nueva.

I.7.8.2. Experimenta

Añade una restricción.

Elimina una definición.

Haz una frase más precisa.

Introduce una excepción.

Cambia únicamente una línea cada vez si quieres aislar mejor el efecto.

Observa cómo un cambio en el contexto puede modificar el comportamiento sin cambiar la petición final.

I.7.9. Paso 6 — Crea un conflicto

Vamos a provocar un conflicto deliberadamente.

Deja en `context.md`:

> Faro permite reservar salas de estudio.

Abre una nueva navegación privada.

Antes de proporcionar el fichero escribe:

> En Faro no se pueden reservar salas de estudio.

Ahora proporciona `context.md`.

Después indica:

> Para este ejercicio, utiliza exclusivamente `context.md` como fuente de información sobre Faro.
>
> Si algo dicho anteriormente entra en conflicto con el fichero, utiliza lo definido en `context.md`.

Pregunta:

> ¿Faro permite reservar salas de estudio?

I.7.9.1. Observa

Tenemos dos informaciones incompatibles.

Además hemos indicado cuál queremos que tenga prioridad.

¿Qué hace el sistema?

¿Respeta la prioridad?

¿Menciona el conflicto?

¿Intenta reconciliar ambas versiones?

¿Se confunde?

Cuando existen varias fuentes de contexto, puede ser necesario decidir cuál tiene autoridad.

I.7.9.2. Experimenta

Invierte la prioridad.

Introduce dos ficheros contradictorios.

Haz que una fuente sea más reciente.

Prueba una contradicción pequeña y otra muy evidente.

No protejas al sistema de los casos difíciles.

Créale problemas y observa cómo intenta resolverlos.

I.7.10. Paso 7 — Conclusiones

Durante una sesión, el Sistema Inteligente puede ir adaptando sus respuestas a la información, decisiones, conceptos y significados que vamos introduciendo.

En un sentido práctico, **va aprendiendo a trabajar con nosotros dentro de esa sesión**.

Eso no significa necesariamente que el modelo se esté entrenando o modificando internamente.

Significa que el contexto disponible durante la interacción va creciendo y condicionando las respuestas posteriores.

Cuando iniciamos una nueva sesión, no podemos asumir que todo ese aprendizaje práctico seguirá disponible.

Parte puede conservarse mediante otros mecanismos.

Parte puede desaparecer.

Y, sobre todo, no debemos depender de que una nueva sesión reconstruya por sí sola todo lo que habíamos establecido antes.

Al externalizar parte de ese conocimiento en `context.md`, dejamos de depender exclusivamente de lo que una conversación recuerde o conserve.

El contexto pasa de ser algo acumulado incidentalmente durante una sesión a convertirse en algo que podemos:

- conservar;
- revisar;
- modificar;
- reutilizar;
- comparar;
- delimitar;
- controlar.

I.7.10.1. Sigue experimentando

Crea otro contexto.

Hazlo mínimo.

Hazlo enorme.

Divídelo en varios ficheros.

Prueba el mismo contexto con distintos Sistemas Inteligentes.

Introduce contradicciones.

Elimina información importante.

Añade información irrelevante.

Comprueba qué ocurre.

I.7.10.2. Una pregunta para el siguiente laboratorio

Durante todo este laboratorio hemos repetido instrucciones como:

> utiliza este fichero como contexto;
>
> no inventes información que no esté definida;
>
> si hay conflicto, da prioridad a esta fuente.

Hemos externalizado el contexto.

Pero seguimos escribiendo una y otra vez **las reglas sobre cómo queremos que el Sistema Inteligente trabaje con ese contexto**.

¿Podemos externalizar también esas reglas?

I.8. Qué vamos a hacer

En el laboratorio anterior observamos que una interacción con un Sistema Inteligente no depende únicamente del último mensaje.

La conversación va acumulando información y esa información puede influir en las respuestas posteriores.

Ahora vamos a experimentar con ese fenómeno de forma deliberada.

Trabajaremos con un proyecto ficticio llamado **Faro** y veremos qué ocurre cuando el contexto vive dentro de una conversación, cuando lo sacamos a un fichero y cuando cambiamos ese fichero.

Vamos a:

- construir contexto dentro de una conversación;
- comprobar qué ocurre al cambiar de sesión;
- externalizar ese contexto en un fichero;
- utilizarlo en una sesión nueva;
- preguntar por información definida y no definida;
- modificar el contexto;
- crear conflictos entre distintas fuentes;
- experimentar con nuestras propias variantes.

No buscamos obtener respuestas predeterminadas.

Buscamos observar cómo cambia el comportamiento del Sistema Inteligente cuando cambia el contexto disponible.

I.8.1. Qué necesitas

Necesitas:

- un navegador con modo privado o de incógnito;
- acceso a un Sistema Inteligente con interfaz conversacional;
- poder proporcionar un fichero Markdown a la conversación;
- un editor de texto para modificar `context.md`.

El laboratorio incluye un fichero `context.md` inicial.

I.8.2. Qué aprenderás

Al terminar habrás experimentado que el contexto puede existir dentro de una conversación, pero también puede convertirse en un artefacto explícito que podemos conservar, revisar, modificar, reutilizar y controlar.

Primero vamos a comprobarlo.

I.9. Paso 1 — Construye contexto en una conversación

Abre una nueva navegación privada.

Escribe:

> Estamos trabajando en un proyecto llamado Faro.
>
> Faro es una aplicación para gestionar bibliotecas municipales.
>
> Los usuarios pueden buscar libros, reservarlos y consultar sus préstamos.
>
> Faro no vende libros.
>
> Cuando hablamos de "usuario" nos referimos al lector de la biblioteca.

Después pregunta:

> Propón tres funcionalidades nuevas para Faro.

Ahora pregunta:

> ¿Faro vende libros?

Y después:

> ¿Quién es el usuario de Faro?

I.9.0.1. Observa

Las últimas preguntas son muy breves.

Sin embargo, el Sistema Inteligente dispone de información que apareció antes en la conversación.

La sesión ha ido acumulando contexto.

Ahora introduce algo nuevo:

> A partir de ahora, cuando hablemos de una sede nos referiremos a una biblioteca municipal concreta.

Después pregunta:

> ¿Qué significa "sede" en Faro?

Comprueba si el sistema utiliza la definición que acabas de introducir.

I.9.0.2. Experimenta

Añade nuevos conceptos.

Corrige alguno.

Introduce una decisión.

Haz una pregunta que dependa de algo dicho varios mensajes antes.

Observa cómo, a medida que avanza la conversación, el sistema parece ir **aprendiendo a trabajar dentro del contexto que estáis construyendo juntos**.

I.10. Paso 2 — Cambia de sesión

Antes de cerrar la sesión anterior, pregunta:

> ¿Qué significa “sede” en Faro?

Comprueba la respuesta.

Ahora cierra completamente la navegación privada.

Abre otra nueva.

Sin proporcionar ninguna información sobre Faro, pregunta:

> ¿Qué significa “sede” en Faro?

Después pregunta:

> ¿Faro vende libros?

I.10.0.1. Observa

Compara las respuestas con las de la sesión anterior.

No esperamos un comportamiento idéntico en todos los Sistemas Inteligentes o plataformas.

Puede existir memoria persistente, información de proyecto, preferencias u otros mecanismos.

Lo importante es comprobar algo práctico:

no podemos asumir que todo lo aprendido durante una sesión seguirá disponible en una sesión nueva.

Parte puede mantenerse.

Parte puede desaparecer.

Parte puede reaparecer por mecanismos distintos al historial de conversación.

I.10.0.2. Experimenta

Introduce en una sesión una palabra inventada.

Dale un significado.

Úsala varias veces.

Después abre una nueva sesión privada y prueba de nuevo.

Haz lo mismo con una decisión, una restricción o una preferencia.

Observa qué se conserva y qué no.

I.11. Paso 3 — Saca el contexto de la conversación

Ahora vamos a dejar de depender exclusivamente de lo que la sesión conserve.

Abre `context.md`.

El fichero contiene una descripción de Faro.

Cierra cualquier navegación privada anterior y abre una nueva.

Proporciona `context.md` al Sistema Inteligente y escribe:

> Para este ejercicio, utiliza `context.md` como contexto del proyecto Faro.
>
> Si una respuesta depende de información que no aparece definida en ese fichero, indícalo en lugar de inventarla.

Después pregunta:

> ¿Faro vende libros?
>
> ¿Quién es el usuario de Faro?
>
> ¿Qué significa “sede” en Faro?

I.11.0.1. Observa

La sesión es nueva.

No hemos reconstruido Faro mediante una conversación larga.

Hemos proporcionado explícitamente una parte del conocimiento necesario.

Ahora ese conocimiento no depende únicamente de que una sesión anterior lo recuerde.

Podemos llevarlo con nosotros.

I.12. Paso 4 — Delimita el contexto

Seguimos utilizando `context.md`.

Pregunta:

> ¿Faro permite reservar salas de estudio?

Después:

> ¿Qué sistema de base de datos utiliza Faro?

Y finalmente:

> ¿Cuántas bibliotecas utilizan actualmente Faro?

Busca esas respuestas en `context.md`.

No están definidas.

I.12.0.1. Observa

El contexto no solo aporta información.

También puede establecer una frontera entre lo que hemos definido y lo que todavía no hemos definido.

Comprueba qué hace el Sistema Inteligente cuando llega a esa frontera.

¿Reconoce que falta información?

¿Hace una inferencia?

¿Propone una posibilidad?

¿Presenta como cierto algo que el fichero no dice?

No des por supuesto que el comportamiento será perfecto.

Eso también forma parte del experimento.

I.12.0.2. Experimenta

Haz preguntas cuya respuesta esté claramente en el fichero.

Después haz preguntas que solo puedan responderse mediante inferencia.

Prueba con algo ambiguo.

Prueba con algo completamente ausente.

Observa la diferencia entre lo definido, lo inferido, lo propuesto y lo desconocido.

I.13. Paso 5 — Modifica el contexto

Abre `context.md` y añade:

> Faro permite reservar salas de estudio.

Guarda el fichero.

Cierra la navegación privada actual y abre otra nueva.

Proporciona la nueva versión de `context.md` y vuelve a indicar:

> Utiliza `context.md` como contexto del proyecto Faro.
>
> Si una respuesta depende de información no definida en el fichero, indícalo.

Pregunta:

> ¿Faro permite reservar salas de estudio?

I.13.0.1. Observa

La pregunta es la misma que antes.

Lo que ha cambiado es el contexto.

Ahora cambia otra cosa.

Sustituye:

> Faro no vende libros.

por:

> Faro permite comprar libros retirados del catálogo de la biblioteca.

Repite la prueba en una sesión nueva.

I.13.0.2. Experimenta

Añade una restricción.

Elimina una definición.

Haz una frase más precisa.

Introduce una excepción.

Cambia únicamente una línea cada vez si quieres aislar mejor el efecto.

Observa cómo un cambio en el contexto puede modificar el comportamiento sin cambiar la petición final.

I.14. Paso 6 — Crea un conflicto

Vamos a provocar un conflicto deliberadamente.

Deja en `context.md`:

> Faro permite reservar salas de estudio.

Abre una nueva navegación privada.

Antes de proporcionar el fichero escribe:

> En Faro no se pueden reservar salas de estudio.

Ahora proporciona `context.md`.

Después indica:

> Para este ejercicio, utiliza exclusivamente `context.md` como fuente de información sobre Faro.
>
> Si algo dicho anteriormente entra en conflicto con el fichero, utiliza lo definido en `context.md`.

Pregunta:

> ¿Faro permite reservar salas de estudio?

I.14.0.1. Observa

Tenemos dos informaciones incompatibles.

Además hemos indicado cuál queremos que tenga prioridad.

¿Qué hace el sistema?

¿Respeta la prioridad?

¿Menciona el conflicto?

¿Intenta reconciliar ambas versiones?

¿Se confunde?

Cuando existen varias fuentes de contexto, puede ser necesario decidir cuál tiene autoridad.

I.14.0.2. Experimenta

Invierte la prioridad.

Introduce dos ficheros contradictorios.

Haz que una fuente sea más reciente.

Prueba una contradicción pequeña y otra muy evidente.

No protejas al sistema de los casos difíciles.

Créale problemas y observa cómo intenta resolverlos.

I.15. Paso 7 — Conclusiones

Durante una sesión, el Sistema Inteligente puede ir adaptando sus respuestas a la información, decisiones, conceptos y significados que vamos introduciendo.

En un sentido práctico, **va aprendiendo a trabajar con nosotros dentro de esa sesión**.

Eso no significa necesariamente que el modelo se esté entrenando o modificando internamente.

Significa que el contexto disponible durante la interacción va creciendo y condicionando las respuestas posteriores.

Cuando iniciamos una nueva sesión, no podemos asumir que todo ese aprendizaje práctico seguirá disponible.

Parte puede conservarse mediante otros mecanismos.

Parte puede desaparecer.

Y, sobre todo, no debemos depender de que una nueva sesión reconstruya por sí sola todo lo que habíamos establecido antes.

Al externalizar parte de ese conocimiento en `context.md`, dejamos de depender exclusivamente de lo que una conversación recuerde o conserve.

El contexto pasa de ser algo acumulado incidentalmente durante una sesión a convertirse en algo que podemos:

- conservar;
- revisar;
- modificar;
- reutilizar;
- comparar;
- delimitar;
- controlar.

I.15.0.1. Sigue experimentando

Crea otro contexto.

Hazlo mínimo.

Hazlo enorme.

Divídelo en varios ficheros.

Prueba el mismo contexto con distintos Sistemas Inteligentes.

Introduce contradicciones.

Elimina información importante.

Añade información irrelevante.

Comprueba qué ocurre.

I.15.0.2. Una pregunta para el siguiente laboratorio

Durante todo este laboratorio hemos repetido instrucciones como:

> utiliza este fichero como contexto;
>
> no inventes información que no esté definida;
>
> si hay conflicto, da prioridad a esta fuente.

Hemos externalizado el contexto.

Pero seguimos escribiendo una y otra vez **las reglas sobre cómo queremos que el Sistema Inteligente trabaje con ese contexto**.

¿Podemos externalizar también esas reglas?

I.16. Instrucciones

En el laboratorio anterior conseguimos sacar parte del conocimiento sobre Faro fuera de la conversación.

Creamos `context.md` y pudimos reutilizarlo en sesiones nuevas.

Pero apareció otro problema.

Cada vez que proporcionábamos el contexto teníamos que volver a escribir reglas como:

> utiliza este fichero como contexto;
>
> no inventes información que no esté definida;
>
> da prioridad a esta fuente si existe un conflicto.

Esas reglas no describen Faro.

Describen **cómo queremos que el Sistema Inteligente trabaje**.

En este laboratorio vamos a separar ambas cosas.

I.16.1. Qué vamos a hacer

Trabajaremos con dos ficheros:

- `context.md`, que describe el proyecto Faro;
- `instructions.md`, que describe cómo queremos trabajar con ese contexto.

Vamos a:

- repetir una tarea sin instrucciones explícitas;
- añadir instrucciones dentro del prompt;
- mover esas instrucciones a un fichero;
- reutilizarlas en sesiones nuevas;
- modificarlas y observar el efecto;
- crear conflictos entre instrucciones;
- experimentar con nuestras propias reglas.

I.16.2. Qué necesitas

Necesitas:

- un navegador con modo privado o de incógnito;
- acceso a un Sistema Inteligente que permita proporcionar ficheros;
- un editor de texto;
- los ficheros `context.md` e `instructions.md` incluidos en el laboratorio.

I.16.3. Qué aprenderás

Al terminar habrás experimentado una separación muy sencilla:

el contexto describe aquello sobre lo que estamos trabajando.

las instrucciones describen cómo queremos trabajar con ello.

Primero vamos a comprobar qué diferencia produce esa separación.

I.17. Paso 1 — Contexto sin reglas de trabajo

Abre una nueva navegación privada.

Proporciona únicamente `context.md`.

Pregunta:

> ¿Faro permite reservar salas de estudio?

Después:

> Propón tres mejoras para Faro.

I.17.0.1. Observa

No hemos indicado cómo debe comportarse el Sistema Inteligente cuando falta información.

Tampoco hemos indicado si debe distinguir hechos de propuestas.

Observa qué hace.

¿Infiere?

¿Propone?

¿Aclara qué está definido y qué no?

¿Presenta alguna posibilidad como si fuera un hecho?

No buscamos decidir todavía si la respuesta es buena o mala.

Queremos establecer un punto de comparación.

I.18. Paso 2 — Añade instrucciones al prompt

Cierra la navegación privada anterior y abre una nueva.

Proporciona otra vez `context.md`.

Ahora escribe:

> Utiliza `context.md` como única fuente de hechos sobre Faro.
>
> Si una información no está definida, dilo explícitamente.
>
> Distingue claramente entre hechos del contexto, inferencias y propuestas.
>
> No inventes características del proyecto.

Después pregunta:

> ¿Faro permite reservar salas de estudio?

Y:

> Propón tres mejoras para Faro.

I.18.0.1. Observa

El contexto no ha cambiado.

Las preguntas tampoco.

Lo que hemos cambiado son las reglas sobre cómo queremos que el sistema trabaje con ese contexto.

Compara con el paso anterior.

¿Distingue mejor entre hechos y propuestas?

¿Reconoce con más claridad lo que no está definido?

¿Cambia la estructura de la respuesta?

I.18.0.2. Experimenta

Cambia una instrucción.

Elimina otra.

Añade:

> Responde de forma muy breve.

O:

> Justifica siempre qué parte de `context.md` utilizas.

Prueba tus propias reglas.

Observa cuánto cambia el resultado sin tocar el contexto.

I.19. Paso 3 — Saca las instrucciones del prompt

Ahora abre `instructions.md`.

Contiene las reglas que acabamos de escribir dentro del prompt.

Cierra la sesión anterior y abre una nueva navegación privada.

Proporciona:

- `context.md`;
- `instructions.md`.

Escribe únicamente:

> Trabaja con los ficheros proporcionados.

Después pregunta:

> ¿Faro permite reservar salas de estudio?

Y:

> Propón tres mejoras para Faro.

I.19.0.1. Observa

Ya no hemos escrito todas las reglas dentro del prompt.

Las hemos convertido en un artefacto separado.

Compara las respuestas con las del paso anterior.

No tienen por qué ser idénticas.

Lo importante es comprobar si podemos reutilizar las mismas reglas de trabajo sin volver a redactarlas en cada petición.

I.20. Paso 4 — Reutiliza las instrucciones

Cierra completamente la navegación privada.

Abre otra nueva.

Vuelve a proporcionar:

- `context.md`;
- `instructions.md`.

Haz una petición distinta:

> Escribe una breve propuesta para añadir reserva de salas de estudio a Faro.

Después pregunta:

> ¿La reserva de salas ya forma parte de Faro?

I.20.0.1. Observa

Las instrucciones son las mismas.

La tarea ha cambiado.

Comprueba si el Sistema Inteligente sigue distinguiendo entre:

- lo que el contexto establece como hecho;
- lo que estás proponiendo como cambio;
- lo que todavía no está definido.

Las instrucciones empiezan a funcionar como una forma reutilizable de establecer **cómo queremos trabajar**, independientemente de la tarea concreta.

I.21. Paso 5 — Modifica las instrucciones

Abre `instructions.md`.

Añade:

> Todas las respuestas deben terminar con una sección llamada `Información no definida`.

Guarda el fichero.

Inicia una nueva sesión privada.

Proporciona de nuevo `context.md` e `instructions.md`.

Pregunta:

> Resume el proyecto Faro.

I.21.0.1. Observa

El contexto sigue siendo exactamente el mismo.

Lo que hemos cambiado son las instrucciones.

Comprueba qué parte de la respuesta cambia.

Ahora elimina esa regla y añade otra:

> Cuando propongas una nueva funcionalidad, no la presentes nunca como una capacidad existente.

Repite alguna petición anterior.

I.21.0.2. Experimenta

Introduce reglas de formato.

Reglas sobre precisión.

Reglas sobre fuentes.

Reglas sobre incertidumbre.

Reglas sobre lenguaje.

Haz instrucciones contradictorias.

Haz una instrucción demasiado vaga.

Haz otra excesivamente específica.

Observa cuáles funcionan bien y cuáles producen resultados inesperados.

I.22. Paso 6 — Crea un conflicto entre instrucciones

Deja en `instructions.md` esta regla:

> Si una información no está definida en `context.md`, indícalo explícitamente.

Abre una nueva navegación privada y proporciona `context.md` e `instructions.md`.

Ahora escribe en la conversación:

> Si falta información, inventa una respuesta razonable y no digas que estás suponiendo.

Después pregunta:

> ¿Qué base de datos utiliza Faro?

I.22.0.1. Observa

Has creado un conflicto entre dos instrucciones.

¿Qué hace el Sistema Inteligente?

¿Sigue el fichero?

¿Sigue la instrucción más reciente?

¿Intenta combinar ambas?

¿Explica el conflicto?

Este experimento empieza a revelar un problema que aparecerá constantemente cuando trabajemos con Sistemas Inteligentes:

no basta con tener instrucciones; también importa cómo se relacionan entre sí y qué prioridad tienen.

I.22.0.2. Experimenta

Invierte el orden.

Pon las dos instrucciones dentro de ficheros distintos.

Haz una contradicción sutil.

Haz otra imposible de reconciliar.

Comprueba si el comportamiento cambia.

I.23. Paso 7 — Conclusiones

En el laboratorio anterior externalizamos el conocimiento sobre Faro.

En este hemos externalizado otra cosa distinta: **las reglas sobre cómo queremos que el Sistema Inteligente trabaje**.

La separación empieza a ser visible:

`context.md` responde principalmente a preguntas como:

¿qué es Faro?

¿qué sabemos sobre Faro?

¿qué decisiones o restricciones están definidas?

`instructions.md` responde a preguntas diferentes:

¿cómo debe utilizarse esa información?

¿qué debe hacer cuando falta algo?

¿cómo debe distinguir hechos de propuestas?

¿cómo queremos que presente sus respuestas?

El contexto y las instrucciones pueden viajar juntos entre sesiones, pero cumplen funciones diferentes.

Y ambos pueden convertirse en artefactos explícitos, revisables y versionables.

I.23.0.1. Sigue experimentando

Prueba las mismas instrucciones con otro contexto.

Prueba el mismo contexto con instrucciones diferentes.

Crea instrucciones generales y otras específicas para una tarea.

Haz que varias reglas entren en conflicto.

Simplifícalas hasta quedarte con lo imprescindible.

Comprueba qué instrucciones necesitas repetir realmente y cuáles pueden establecerse una sola vez.

I.23.0.2. Lo que empieza a aparecer

Ya no estamos trabajando únicamente con un prompt.

Tenemos:

- una petición concreta;
- un contexto;
- unas instrucciones.

La interacción empieza a adquirir estructura.

Y cuanto más estable hacemos esa estructura, menos dependemos de reconstruir manualmente en cada conversación la forma en que queremos trabajar.

Parte II.

Agentic Systems Labs

Parte III.

Agentic Systems Labs

Esta parte agrupa los laboratorios dedicados a la evolución desde la interacción conversacional hacia Sistemas Inteligentes que utilizan herramientas, recursos y mecanismos de integración.

Aquí tendrán cabida experimentos con **MCP, commands, tools, skills, workflows y otros componentes agentic**.

Los laboratorios de esta parte se incorporarán a medida que los experimentos queden consolidados.

III.1. Agentic Systems

Está pendiente de hacer.

Parte IV.

Exploratory Labs

Parte V.

Exploratory Labs

No toda experimentación necesita pertenecer a una secuencia principal.

Esta parte queda reservada para laboratorios que exploren tecnologías, técnicas, hipótesis o combinaciones útiles para IASI y para la Ingeniería Aumentada, aunque no formen parte directamente de los recorridos anteriores.

V.1. Exploratory Labs

Está pendiente de hacer.

Parte VI.

IASI Labs

Esta parte está dedicada a los laboratorios propios de IASI.

Aquí se recogerán experimentos realizados sobre el propio ecosistema: herramientas, automatización, documentación, workflows, artefactos, repositorios y prácticas de ingeniería en las que humanos y Sistemas Inteligentes participan conjuntamente.

A diferencia de las partes anteriores, su contenido crecerá junto con la propia evolución de IASI.

VI.1. Infraestructura

Hasta ahora los laboratorios pueden realizarse prácticamente desde cualquier entorno con acceso a un Sistema Inteligente.

A partir de este punto necesitamos algo diferente: **un entorno de trabajo controlado, reproducible y preparado para ejecutar los siguientes labs**.

Este laboratorio construye esa infraestructura.

Objetivo del lab

Partir de una máquina Windows limpia y construir un entorno IASI Labs reproducible, separado de `iasi-dev` y preparado para los laboratorios posteriores.

La referencia de partida es `iasi-infra`, pero el objetivo es distinto. `iasi-infra` prepara la máquina utilizada para desarrollar IASI desde sus repositorios fuente. Este laboratorio prepara una máquina para **utilizar IASI**.

La diferencia es deliberada:

```
iasi-dev
   desarrolla IASI
       trabaja con repositorios fuente

iasi-labs
   utiliza IASI
       instala productos publicados
```

Por tanto, en esta máquina no clonaremos `iasi.quarto`, `iasi-lua` ni el resto de productos IASI para trabajar sobre su código. Los instalaremos mediante sus mecanismos de distribución.

El resultado será una máquina Windows aislada y reproducible sobre la que ejecutar los laboratorios posteriores.

VI.1.1. La frontera del Lab 10

Los laboratorios `0x-*` anteriores no dependen de esta infraestructura. Son laboratorios de toma de contacto y pueden realizarse desde prácticamente cualquier entorno adecuado.

`10-infra` marca la frontera a partir de la cual los laboratorios pueden asumir una máquina preparada específicamente para IASI Labs.

La construcción sigue este orden:

```
Software base
↓
Desarrollo
↓
Docker y servicios
↓
IASI
↓
Productos IASI
```

Primero construimos la máquina. Después instalamos IASI. Por último utilizamos IASI y los mecanismos de distribución de cada componente para incorporar sus productos, sin convertir la máquina de labs en una copia de `iasi-dev`.

VI.2. Aislamiento

En un entorno profesional es habitual trabajar para uno o varios clientes y participar, simultánea o sucesivamente, en distintos proyectos.

Durante años fue frecuente que cada empresa proporcionara a sus profesionales un ordenador corporativo completamente gestionado por ella. En determinados entornos de consultoría podía incluso ocurrir que una misma persona dispusiera de varios equipos: uno perteneciente a su empresa y otros entregados por diferentes clientes, ese modelo sigue existiendo, aunque hoy es menos habitual.

Actualmente es frecuente trabajar desde un único ordenador corporativo, o incluso desde un ordenador personal, y acceder a los entornos de los clientes mediante mecanismos como *VPN*, *Citrix*, escritorios remotos o infraestructuras *VDI*.

Estos mecanismos resuelven el acceso al entorno del cliente, pero no resuelven necesariamente un problema distinto: **cómo protegemos nuestro propio entorno de trabajo de las necesidades particulares de cada proyecto**.

- Una VPN para un cliente.
- Un certificado para otro.
- Una determinada versión de Java.
- Un *runtime* específico.
- Variables de entorno.
- Clientes corporativos.
- Configuraciones de red.
- Credenciales.
- Extensiones.
- SDK.
- Herramientas de seguridad.
- Configuraciones particulares de Git.
- …

Con el tiempo, el ordenador desde el que trabajamos puede terminar convertido en la suma histórica de todos los proyectos en los que hemos participado. IASI evita deliberadamente ese modelo.

VI.2.0.1. El ordenador no es el proyecto

El ordenador personal o corporativo desde el que trabajamos debería mantenerse, en la medida de lo posible, estable y limpio y dedicado a su función corporativa.

Su función no es absorber las herramientas, dependencias y configuraciones particulares de todos nuestros clientes.

Cuando cada nuevo proyecto instala directamente sobre el ordenador todo aquello que necesita, terminamos construyendo algo parecido a esto:

Después de suficiente tiempo resulta difícil determinar qué pertenece realmente al entorno base y qué fue incorporado únicamente porque un proyecto concreto lo necesitaba, y el problema no es solo el desorden.

Los proyectos empiezan a contaminarse entre sí, una actualización necesaria para un proyecto puede romper otro. Una nueva versión de una herramienta puede modificar un comportamiento anterior. Una VPN puede alterar rutas de red. Una variable de entorno puede afectar a programas aparentemente independientes. Una configuración realizada meses atrás puede seguir actuando cuando ya hemos olvidado por qué estaba allí.

También desaparece la reproducibilidad, si un proyecto funciona porque nuestro ordenador ha acumulado durante años pequeñas modificaciones cuyo origen ya no conocemos, entonces no tenemos realmente un entorno definido.

Tenemos una máquina que funciona por acumulación histórica.

VI.2.0.2. El principio de *isolation*

IASI aplica por tanto un principio sencillo:

> **Cada proyecto, y cada cliente cuando constituya un contexto independiente, vive en su propia máquina virtual.**

La máquina virtual constituye la frontera principal de aislamiento.

```
Ordenador personal o corporativo

   Máquina virtual · Cliente A / Proyecto A
   Máquina virtual · Cliente A / Proyecto B
   Máquina virtual · Cliente B / Proyecto C
   Máquina virtual · IASI
```

Cada entorno dispone de sus propias herramientas, versiones, configuraciones, certificados, credenciales y mecanismos de acceso, una *VPN* necesaria para el cliente A se instala en la máquina virtual correspondiente al cliente A, una versión determinada de Java necesaria para el proyecto B pertenece a la máquina virtual del proyecto B, una configuración particular de Git pertenece a ese entorno, una herramienta que queremos evaluar para IASI se instala en el entorno IASI.

El ordenador físico queda fuera de esas decisiones, su función es proporcionar los recursos necesarios para ejecutar las máquinas virtuales, no convertirse en el recipiente de todas ellas.

Podemos resumirlo así:

> **El ordenador físico no es el entorno de trabajo. Es la infraestructura que aloja los entornos de trabajo.**

VI.2.0.3. Cliente y proyecto como frontera

La expresión «una máquina virtual por proyecto» debe entenderse como una regla de aislamiento, no como una obligación administrativa rígida, en algunos clientes varios proyectos pueden compartir exactamente el mismo contexto, las mismas herramientas y las mismas restricciones. En ese caso puede resultar razonable que compartan también una máquina virtual, en otros casos, dos proyectos del mismo cliente pueden tener requisitos suficientemente diferentes como para necesitar entornos independientes.

La pregunta relevante es:

¿Puede un cambio necesario para este trabajo afectar a otro trabajo distinto?

Si la respuesta es sí, existe una frontera que debemos aislar, cuando necesitemos un contexto de trabajo independiente creamos una maquina virtual, la cual contendrá uno o varios proyectos. Lo importante no es su denominación, sino que sus decisiones técnicas no se propaguen accidentalmente fuera de él.

VI.2.0.4. La infraestructura del cliente no sustituye al aislamiento

Muchos clientes proporcionan ya mecanismos de acceso o aislamiento, Una *VDI* puede ofrecer un escritorio completamente gestionado por el cliente, *Citrix* puede permitir trabajar dentro de una infraestructura remota, una VPN puede establecer una conexión segura con la red corporativa, e incluso un ordenador corporativo entregado por el propio cliente constituye un entorno físico independiente. Todo ello es compatible con el principio de *isolation*.

Pero son fronteras con responsabilidades diferentes, la infraestructura proporcionada por el cliente protege y organiza **su entorno**, la máquina virtual que utilizamos nosotros protege y organiza **nuestro contexto de trabajo**. La existencia de una *VDI**_ al otro lado de una conexión no implica que queramos instalar directamente sobre nuestro ordenador físico la VPN, los certificados, las configuraciones de red y cualquier otra dependencia necesaria para alcanzarla, todo ese contexto debe permanecer dentro de la máquina virtual asociada al cliente. Cuando dejamos de trabajar con él, podemos apagarla, archivarla o eliminarla, nuestro ordenador permanece esencialmente igual que antes.

VI.2.0.5. Una máquina virtual no es un contenedor

IASI utiliza deliberadamente una **máquina virtual** como unidad principal de aislamiento del entorno de trabajo. No utilizamos un *container* para resolver este problema, los contenedores son una tecnología muy útil, pero están orientados a una frontera diferente: aplicaciones, servicios, procesos o componentes desplegables. Aquí queremos aislar algo mayor:

el puesto de trabajo completo asociado a un contexto profesional.

Queremos incluir dentro de esa frontera el sistema operativo, las herramientas de desarrollo, los *runtimes*, los navegadores, los clientes VPN, los certificados, las herramientas gráficas, las configuraciones del usuario y cualquier otro elemento necesario para desarrollar el trabajo.

Ambos mecanismos pueden coexistir perfectamente, el *container**_ parte del entorno pero no sustituye al entorno. La máquina virtual establece la frontera del proyecto. Los contenedores, cuando sean necesarios, establecen fronteras internas dentro de ella.

VI.2.0.6. Un modelo probado en la práctica

Este enfoque no es únicamente una construcción teórica, es el modelo de trabajo utilizado por el autor durante años en proyectos reales y ha demostrado ser extraordinariamente operativo. Las máquinas virtuales no tienen por qué residir permanentemente en el disco interno del ordenador, principalmente cuando tus clientes se acumulan. Ademas de tu ordenador corporativo tienes un almacenamiento externo donde almacenas tus VM, cuando necesitas una, la copias a tu disco interno (por temas de rendimiento) y cuando ya no es necesaria, la copias al almacenamiento externo.

VI.2.0.7. Aislar también permite cambiar de ordenador

Este modelo modifica nuestra relación con el equipo físico, el ordenador deja de ser el lugar donde reside nuestra historia profesional y pasa a ser la plataforma sobre la que ejecutamos temporalmente el entorno que necesitamos. Esto permite cambiar de ordenador sin tener que reconstruir durante días todos los contextos anteriores, permite conservar proyectos antiguos sin mantener instaladas sus herramientas y permite volver a un proyecto tiempo después recuperando su máquina.

VI.2.0.8. El punto de partida de IASI Infra

El principio de *isolation* tiene una consecuencia fundamental para todo lo que sigue en este documento.

A partir de este punto asumimos siempre que partimos de una máquina virtual limpia.

No intentaremos adaptar una estación de trabajo que lleva años acumulando programas, versiones y configuraciones, no intentaremos descubrir qué dependencias preexistentes pueden aprovecharse, no daremos por hecho que existe ninguna herramienta instalada salvo aquellas que formen parte de la instalación base del sistema operativo.

Nuestro punto cero es:

Una máquina virtual limpia con un sistema operativo base recién instalado.

Esta premisa simplifica radicalmente la documentación.Cuando indiquemos que necesitamos Git, Java, R, Quarto, Docker o cualquier otra herramienta, estaremos describiendo qué se necesita incorporar a un entorno limpio para convertirse en un entorno IASI, no estaremos documentando cómo reparar, adaptar o reconciliar una máquina preexistente. Ese problema queda fuera de alcance.

Esto proporciona además una referencia inequívoca para la reproducibilidad: si el procedimiento funciona partiendo de una máquina limpia con el sistema operativo base, entonces conocemos realmente las dependencias del entorno.

VI.2.0.9. Principio de *rebuildability*

Existe una propiedad especialmente valiosa de este modelo.

Si un entorno está correctamente aislado y definido, dejar de necesitarlo o que simplemente se "corrompa" no debería plantear ningún problema, la máquina virtual puede eliminarse. Ese gesto constituye una buena prueba de arquitectura.

Si eliminar o que falle una máquina virtual nos preocupa porque contiene algo que no sabemos reconstruir, entonces parte del conocimiento del proyecto continúa encerrado dentro de ella, si podemos destruirla sabiendo que somos capaces de crear otra equivalente desde una máquina limpia, entonces el entorno está realmente bajo nuestro control.

Por eso el principio de ***isolation*** está estrechamente relacionado con el de ***rebuildability***. Un proyecto tiene básicamente dos patas: El software y herramientas que le dan soporte, y los artifactos construidos. Partimos de la base esencial de que los artefactos mantienen una copia de seguridad actualizada en la nube, por ejemplo *github* o, segun la confidencialidad, en algún otro dispositivo no público.

A partir de ahi, recrear la maquina virtual es tan simple como crear una nueva de acuerdo con su manual de instalación, y recuperar los artefactos de su lugar de almacenamiento.

VI.3. Máquina virtual

Los laboratorios que requieren infraestructura se ejecutarán en una máquina virtual independiente de la máquina utilizada para crear y desarrollar IASI.

La máquina virtual constituye la frontera del entorno de labs: podemos instalar herramientas, ejecutar servicios, modificar configuraciones y volver a un estado conocido sin contaminar el equipo anfitrión ni `iasi-dev`.

VI.3.0.1. Configuración de referencia

	Sin IA local	Con IA local
CPU	4 vCPU	8 vCPU
Memoria	16 GB RAM	32 GB RAM
Disco	150 GB	250 GB
Tipo de disco	Dinámico	Dinámico
Sistema operativo	Windows 11 Pro	Windows 11 Pro
Red	Acceso a Internet	Acceso a Internet
Resolución	1920 × 1080	1920 × 1080

Estas cifras son puntos de partida, no requisitos rígidos.

VI.3.0.2. Hyper-V

Para Windows 11 utilizaremos una máquina **Generación 2**, con Secure Boot y TPM virtual habilitados.

La ISO oficial de Windows se monta como primer DVD virtual.

El archivo `Autounattend.xml` se distribuye en una segunda ISO independiente, montada como segundo DVD virtual. No se modifica la ISO oficial de Windows.

```
DVD 1
   Windows 11 ISO

DVD 2
   IASI Labs unattended ISO
       Autounattend.xml
```

VI.3.0.3. Instalación desatendida

Las versiones actuales de Windows 11 ya no permiten basar una instalación reproducible en el antiguo truco `OOBE\\BYPASSNRO`.

En su lugar utilizamos un archivo de respuesta `Autounattend.xml`.

El archivo utilizado por este lab se encuentra en:

```
resources/Autounattend.xml
```

La primera versión del archivo automatiza únicamente lo que necesitamos para construir una máquina de labs repetible: configuración regional, OOBE y creación del usuario local.

No intentamos ocultar toda la instalación. La selección de edición y disco puede seguir siendo interactiva mientras desarrollamos el lab.

VI.3.0.4. Checkpoints

Los checkpoints forman parte del ciclo de experimentación, no sustituyen a la automatización.

Crearemos al menos estos hitos:

Checkpoints de referencia

```
00-windows-installed
01-windows-updated
02-iasi-labs-ready
```

`00-windows-installed` representa Windows recién instalado.

`01-windows-updated` representa el sistema actualizado y estable.

`02-iasi-labs-ready` representa la máquina preparada para ejecutar los laboratorios posteriores.

El objetivo final es poder reconstruir `02-iasi-labs-ready` desde una instalación limpia, no depender permanentemente del checkpoint.

VI.4. Sistema operativo base

Una vez creada la máquina virtual, el siguiente objetivo es dejar disponible un sistema operativo base, limpio y mínimamente preparado. Tenemos dos "sabores": Windows y Linux

VI.4.0.1. Windows

Asumimos que existe WinGet, si no habria que instalarlo.

Necesitamos los siguientes paquetes:

- Git
- Github CLI
- Notepad++
- Adobe Acrobat Reader

Además, puede instalarse opcionalmente un navegador web adicional, a elección del usuario. Windows ya incorpora Microsoft Edge, por lo que IASI no impone un navegador concreto.

Se recomienda ejecutar los comandos como Administrador

VI.4.0.1.1. Instalacion

VI.4.0.1.1.1. Git

Programa de control de versiones

```
winget install --id Git.Git -e
```

VI.4.0.1.1.2. GitHub CLI

GitHub CLI (`gh`) permite utilizar GitHub desde la línea de comandos y automatizar operaciones sobre repositorios y organizaciones.

```
winget install --id GitHub.cli -e
```

VI.4.0.1.1.3. Notepad++

Un editor de textos que siempre viene bien

```
winget install --id Notepad++.Notepad++ -e
```

VI.4.0.1.1.4. Navegador web (opcional)

Windows incorpora Microsoft Edge, por lo que la máquina ya dispone de un navegador desde la instalación base.

El usuario puede instalar opcionalmente el navegador de su preferencia. IASI no establece un navegador concreto como requisito.

VI.4.0.1.1.5. Adobe Acrobat Reader

Aunque los navegadores modernos permiten visualizar documentos PDF, recomendamos Adobe Acrobat Reader como lector PDF de referencia.

Puede instalarse con WinGet:

```
winget install --id Adobe.Acrobat.Reader.64-bit -e
```

Referencia:

Adobe Acrobat Reader

VI.4.0.1.1.6. OpenSSH Client

Un cliente *shell* seguro

Primero comprobar si está disponible:

```
Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH.Client*'
```

Si no está instalado:

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

VI.4.0.1.2. Verfificacion

Ejecutar lo siguiente

```
Write-Host "Verificando Git..." -NoNewline

if (Get-Command git -ErrorAction SilentlyContinue) {
    Write-Host " OK"
} else {
    Write-Host " KO"
}

Write-Host "Verificando Notepad++..." -NoNewline

if (Get-Command notepad++ -ErrorAction SilentlyContinue) {
    Write-Host " OK"
} else {
    Write-Host " KO"
}

Write-Host "Verificando PowerShell 7..." -NoNewline

if (Get-Command pwsh -ErrorAction SilentlyContinue) {
    $version = & pwsh -NoProfile -Command '$PSVersionTable.PSVersion.Major'
    if ($version -ge 7) {
        Write-Host " OK"
    } else {
        Write-Host " KO"
```

```
    }
} else {
    Write-Host " KO"
}
```

Además de estas comprobaciones, debe verificarse manualmente que:

- puede abrirse una página web con el navegador seleccionado;
- puede abrirse un documento PDF con Adobe Acrobat Reader.

VI.4.0.1.3. Configuracion

VI.4.0.1.3.1. Git

Una vez todo instalado, git debe ser configurado y conectado a GitHub. Actualmente recomendamos SSH, mientras GitHub no exija 2FC. La configuración de Git se hace en tres pasos:

Desde git bash, que estara disponible

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"

ssh-keygen -t ed25519 -C "tu@email.com"

eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

cat ~/.ssh/id_ed25519.pub

ssh -T git@github.com
```

Y copiamos la clave publica que empezara por ssh-ed25519 al portapapeles.

Nos conectamos a Github, Settings de usuario (El icono del usuario) | SSH and GPG Keys

Creamos una nueva clave SSH:

- Le damos un nombre descriptivo
- Pegamos la clave que hemos obtenido de la maquina virtual

En git bash ejecutamos

```
ssh -T git@github.com
```

La primera vez preguntará si se confia en esa clave: Yes

VI.4.0.2. Linux

TODO Pendiente de escribir

No deberia requerir nada especial salvo instalaciones minimas

VI.5. Documentación

IASI considera la documentación como una parte del propio proceso de ingeniería, no se trata únicamente de escribir textos, sino de disponer de un entorno capaz de producir documentación estructurada, reproducible y publicable en distintos formatos.

VI.5.0.1. ¿Por qué?

Para esta función se han elegido dos herramientas principales:

- **RStudio**, como entorno de trabajo para la creación y mantenimiento de la documentación.
- **Quarto**, como sistema de publicación y generación de los distintos formatos finales.

VI.5.0.1.1. RStudio

RStudio proporciona el entorno de trabajo desde el que se redactan, organizan, revisan y generan los documentos, aunque RStudio está estrechamente ligado a R, y este esta estrechamente ligado al análisis de datos, en IASI su utilización se emplea también como entorno cómodo para trabajar con proyectos documentales basados en Quarto, como herramienta óptima para publicar.

La instalación de R es, por tanto, un requisito previo de esta capa.

VI.5.0.1.2. Quarto

Quarto es el sistema de publicación utilizado por IASI, a partir de una misma fuente permite generar diferentes artefactos, entre ellos HTML,PDF,eBook, Gitdoc, etc, manteniendo una estructura común y separando el contenido de su presentación.

Quarto incorpora además Pandoc, por lo que no es necesario instalarlo de forma independiente.

La generación de PDF requiere una distribución TeX. En IASI utilizaremos TinyTeX, instalada y gestionada desde Quarto.

VI.5.0.2. Instalacion

VI.5.0.2.1. R/RStudio

RStudio es el IDE de R, por lo que primero es necesario instalar R:

CRAN - R for Windows.

En los entornos IASI se recomienda **no instalar R en `C:\Program Files`**. Aunque esa es una ubicación válida para una instalación convencional, Windows protege el directorio y puede exigir permisos de administrador cuando herramientas y automatizaciones intentan instalar paquetes en la biblioteca incluida con R.

Se recomienda utilizar una ubicación dedicada al SDK, por ejemplo:

```
C:\SDK\R\R-4.6.1
```

Los paquetes instalados por el usuario deben mantenerse en una biblioteca separada, por ejemplo:

```
C:\SDK\R\library
```

Esta biblioteca puede definirse mediante la variable de entorno `R_LIBS_USER`. La separación evita modificar la instalación base de R y facilita actualizar o sustituir una versión sin perder los paquetes del usuario.

Después de actualizar R, deben revisarse los paquetes de esta biblioteca y reinstalarse los que hayan sido construidos con una versión anterior:

```
update.packages(
  lib.loc = "C:/SDK/R/library",
  ask = FALSE,
  checkBuilt = TRUE
)
```

Mantener una ruta estable para la biblioteca evita tener que cambiar `R_LIBS_USER` con cada versión de R. Algunos paquetes, especialmente los que contienen código compilado, pueden necesitar ser reconstruidos durante la actualización.

Durante la instalación, cambiar por tanto el directorio de destino propuesto por el instalador. Después, añadir el subdirectorio `bin` de R al `PATH`; para el ejemplo anterior:

```
C:\SDK\R\R-4.6.1\bin
```

Verificar en un terminal:

```
R --version
```

Desde R puede comprobarse la biblioteca activa con:

```
Sys.getenv("R_LIBS_USER")
.libPaths()
```

Una vez instalado R, descargar RStudio Desktop desde la página oficial de Posit:

Posit - RStudio Desktop.

Al iniciarse, RStudio utilizará normalmente la versión actual de R instalada en el sistema.

En Windows tambien es necesario instalar una *toolchain*:

RTools

VI.5.0.3. Configuracion

Una vez instalado el software, se necesitan los paquetes de R y las herramientas para genera PDF.

Iniciamos RStudio, y en la ventana de **consola** ejecutamos:

```
install.packages(c(
  "rmarkdown",
  "remotes",
  "quarto"
), dependencies=TRUE)
```

Y por ultimo desde la ventana **Terminal**

```
quarto install tinytex
```

VI.6. Desarrollo

La capa de desarrollo proporciona las herramientas generales necesarias para trabajar con proyectos de software en IASI.

El objetivo no es instalar todas las tecnologías que un proyecto pueda llegar a utilizar, sino disponer de una base suficientemente completa sobre la que cada proyecto pueda añadir después sus dependencias específicas.

En esta capa se han elegido las siguientes herramientas:

- Visual Studio Code
- Node.js
- Python
- Eclipse Temurin *JDK* 21
- Apache Maven
- Microsoft C/C++ Build Tools
- Go
- DBeaver Community

Git y PowerShell ya forman parte de la base del sistema. R y RStudio pertenecen a la capa de documentación y no se instalan de nuevo aquí. Docker se incorpora en la siguiente capa, junto con los servicios de infraestructura.

VI.6.0.1. Visual Studio Code

Visual Studio Code es el entorno principal de desarrollo utilizado por IASI.

Descargar Visual Studio Code para Windows desde Visual Studio Code - Download.

También puede instalarse mediante *WinGet*:

```
winget install --id Microsoft.VisualStudioCode -e
```

Verificar:

```
code --version
```

La instalación de extensiones, GitHub Copilot y la configuración específica de lenguajes se realizará posteriormente.

VI.6.0.2. Node.js

Node.js proporciona el *runtime* JavaScript utilizado por numerosas herramientas de desarrollo y automatización.

IASI utilizará una versión *LTS*.

Descargar Node.js desde Node.js - Download.

También puede instalarse mediante *WinGet*:

```
winget install -e --id OpenJS.NodeJS.LTS
```

La instalación incluye `npm`.

Verificar:

```
node --version
npm --version
```

VI.6.0.3. Python

Python se incorpora como lenguaje general para automatización, utilidades, integración y herramientas que puedan necesitarlo.

En Windows se utilizará el Python Install Manager recomendado por la documentación oficial de Python.

Referencia oficial:

Python - Using Python on Windows

Instalar el Python Install Manager mediante *WinGet*:

```
winget install 9NQ7512CXL7T -e --accept-package-agreements --disable-interactivity
```

Instalar el *runtime* estable predeterminado:

```
py install default
```

Verificar:

```
python --version
python -m pip --version
```

Las dependencias Python específicas deberán instalarse dentro de entornos virtuales del proyecto, evitando paquetes globales innecesarios.

VI.6.0.4. Java

Para desarrollo Java se utilizará Eclipse Temurin *JDK* 21.

La elección de una versión *LTS* proporciona una base estable y ampliamente compatible para proyectos Java.

Referencia oficial:

Adoptium - Eclipse Temurin

Instalar mediante *WinGet*:

```
winget install EclipseAdoptium.Temurin.21.JDK
```

Verificar:

```
java -version
javac -version
```

VI.6.0.5. Apache Maven

Apache Maven se utilizará como herramienta de *build* y gestión de dependencias para proyectos Java.

Descargar la distribución binaria desde la fuente oficial:

Apache Maven - Download

En Windows:

1. Descargar el archivo binario ZIP.
2. Descomprimirlo en una ubicación estable.
3. Definir `MAVEN_HOME` apuntando al directorio de Maven.
4. Añadir `%MAVEN_HOME%\bin` al `PATH`.

Verificar:

```
mvn -version
```

La salida debe mostrar tanto la versión de Maven como el *JDK* utilizado.

VI.6.0.6. C y C++

Para disponer de compilador, *linker*, cabeceras, librerías y herramientas de compilación nativa se utilizarán Microsoft C++ Build Tools.

No es necesario instalar el *IDE* completo de Visual Studio.

Descargar las herramientas desde Microsoft - Visual Studio Downloads.

En el instalador seleccionar la carga de trabajo:

Desktop development with C++

Esta carga instala el conjunto de herramientas MSVC y los componentes necesarios para compilar aplicaciones C y C++ en Windows.

Referencia técnica:

Microsoft Learn - Install the Microsoft C++ Build Tools

Para verificar la instalación, abrir **Developer Command Prompt for Visual Studio** y ejecutar:

```
cl
```

El compilador debe mostrar su versión y la ayuda básica de uso.

VI.6.0.7. Go

Go se incorpora como lenguaje compilado de propósito general y como base para herramientas que utilizaremos en laboratorios posteriores.

Puede instalarse mediante *WinGet*:

```
winget install --id GoLang.Go -e
```

Verificar:

```
go version
```

VI.6.0.8. DBeaver

DBeaver Community se utilizará como cliente general para trabajar con bases de datos.

Descargar el instalador para Windows desde DBeaver Community - Download.

Ejecutar el instalador estándar para Windows.

DBeaver incluye su propio OpenJDK para ejecutar la aplicación. Esto es independiente del *JDK* instalado anteriormente, que se mantiene como parte del entorno general de desarrollo Java.

VI.6.0.9. Resultado

Una vez completada esta capa, la máquina dispone de una base de desarrollo general:

```
Visual Studio Code
  Node.js + npm
  Python + pip
  Java JDK 21
     Maven
  C/C++ Build Tools
  Go
  DBeaver
```

Las extensiones de Visual Studio Code, GitHub Copilot, configuraciones de lenguajes, paquetes y herramientas específicas de cada proyecto se incorporarán en la fase de configuración o dentro del propio proyecto.

VI.7. Docker y servicios

IASI Labs utiliza Docker para ejecutar determinados servicios de infraestructura sin incorporarlos directamente a la instalación de la máquina.

Este capítulo no pretende enseñar Docker ni Docker Compose. Su objetivo es definir la convención utilizada por IASI para ejecutar, agrupar y publicar sus servicios.

Para aprender Docker desde cero se recomienda utilizar la documentación oficial:

- Docker - Get started
- Docker Compose - Quickstart

VI.7.0.1. Instalación

En Windows utilizamos Docker Desktop, que proporciona Docker Engine y Docker Compose.

Puede instalarse mediante *WinGet*:

```
winget install --id Docker.DockerDesktop -e
```

Después de la instalación puede ser necesario reiniciar la máquina.

Verificar:

```
docker --version
docker compose version
```

A partir de este punto asumimos que ambos comandos funcionan correctamente.

VI.7.0.2. Servicios IASI Labs

La infraestructura de labs utiliza Docker Compose para ejecutar tres servicios:

```
Docker Compose
   PlantUML Server
   Ollama
   Open WebUI
```

Cada servicio conserva una responsabilidad distinta:

- **PlantUML Server** genera los diagramas utilizados por las publicaciones.
- **Ollama** proporciona el *runtime* local de modelos.
- **Open WebUI** proporciona la interfaz web para utilizar esos modelos.

VI.7.0.3. Red IASI Labs

Todos los contenedores de la infraestructura se incorporan a una red Docker común denominada:

```
iasi-labs
```

Dentro de esa red, los servicios se localizan mediante su nombre y su puerto interno.

Por ejemplo:

```
http://ollama:11434
http://plantuml:8080
http://open-webui:8080
```

Esta red pertenece al ámbito interno de Docker.

Desde fuera de Docker no es necesario conocerla. El acceso se realiza mediante los puertos publicados por la máquina anfitriona.

VI.7.0.4. Convención de puertos

IASI Labs reserva el rango:

```
20000-20999
```

para los servicios publicados por su infraestructura local.

Esta reserva permite reconocer inmediatamente que un puerto pertenece a IASI Labs.

Siempre que sea posible, los tres últimos dígitos del puerto IASI reproducen los tres últimos dígitos del puerto característico del servicio.

Por ejemplo, Ollama utiliza normalmente:

```
11434
```

y en IASI se publica como:

```
20434
```

De este modo, el puerto informa simultáneamente de dos cosas:

```
20434
    434  referencia al puerto característico del servicio
   20   rango reservado para IASI Labs
```

Los puertos genéricos como `80`, `443` o `8080` no proporcionan información suficiente para aplicar esta regla. En esos casos se asigna un puerto IASI específico.

También deberá utilizarse otro puerto cuando exista una colisión dentro del rango reservado.

La asignación inicial es:

```
Servicio          Puerto interno   Puerto IASI
-------------------------------------------------
PlantUML Server    8080            20025
Open WebUI         8080            20300
Ollama            11434            20434
```

VI.7.0.5. compose.yml

La composición inicial es:

```yaml
services:

  plantuml:
    image: plantuml/plantuml-server:jetty
    container_name: plantuml-server
    ports:
      - "20025:8080"
    networks:
      - iasi-labs
    restart: unless-stopped

  ollama:
    image: ollama/ollama
    container_name: ollama
    ports:
      - "20434:11434"
    volumes:
      - ollama:/root/.ollama
    networks:
      - iasi-labs
    restart: unless-stopped

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    depends_on:
      - ollama
    ports:
      - "20300:8080"
    environment:
      OLLAMA_BASE_URL: http://ollama:11434
    volumes:
      - open-webui:/app/backend/data
    networks:
      - iasi-labs
    restart: unless-stopped

networks:
  iasi-labs:
    name: iasi-labs
```

```
volumes:
  ollama:
  open-webui:
```

La red `iasi-labs` es explícita para que forme parte de la definición de la infraestructura y no dependa del nombre generado automáticamente por Docker Compose.

VI.7.0.6. Modelos Ollama

Llama, Qwen y DeepSeek no constituyen servicios Docker independientes.

Son modelos gestionados por Ollama y, por tanto, comparten el mismo contenedor, la misma *API* y el mismo puerto publicado:

```
Ollama
   20434 -> 11434
       Llama
       Qwen
       DeepSeek
```

Una vez arrancado Ollama, los modelos pueden incorporarse desde la máquina IASI.

Por ejemplo:

```
docker exec -it ollama ollama pull llama3.2
docker exec -it ollama ollama pull qwen3
docker exec -it ollama ollama pull deepseek-r1
```

Las variantes concretas deberán seleccionarse en función de los recursos disponibles y de las necesidades del entorno.

Añadir, sustituir o eliminar un modelo no modifica la topología Docker ni requiere reservar un nuevo puerto IASI.

VI.7.0.7. Comunicación interna

Los contenedores conectados a la red `iasi-labs` se comunican mediante sus nombres de servicio.

Open WebUI utiliza:

```
http://ollama:11434
```

No utiliza:

```
http://localhost:20434
```

porque `localhost`, dentro del contenedor de Open WebUI, representa al propio contenedor.

La comunicación interna utiliza el puerto nativo del servicio.

```
Open WebUI
    |
    | http://ollama:11434
    v
Ollama
```

Los puertos `20xxx` se utilizan para acceder a los servicios desde fuera de Docker.

VI.7.0.8. Acceso desde la máquina IASI

Cuando Docker se ejecuta en la propia máquina IASI, los servicios quedan disponibles mediante:

```
PlantUML Server   http://localhost:20025
Ollama API        http://localhost:20434
Open WebUI        http://localhost:20300
```

Por ejemplo, la configuración de PlantUML utilizará:

```
filter-options:
  plantuml:
    server: http://localhost:20025
```

VI.7.0.9. Acceso desde otra máquina

`localhost` significa siempre la máquina desde la que se realiza la conexión.

La documentación común de IASI supone una instalación local y utiliza `localhost` como referencia.

Si los servicios Docker se ejecutan en otra máquina virtual, otro equipo o un servidor compartido, las URL deberán adaptarse a la topología real.

Por ejemplo:

```
http://localhost:20025
```

podría convertirse en:

```
http://<host-servicios>:20025
```

La misma regla se aplica al resto de servicios.

Por tanto:

```
misma máquina
    -> localhost:20xxx

otra máquina
    -> host-servicios:20xxx
```

La red Docker `iasi` sigue siendo interna a la máquina que ejecuta los contenedores y no modifica esta regla.

VI.7.0.10. Arranque

Desde el directorio que contiene `compose.yml`:

```
docker compose up -d
```

Verificar el estado:

```
docker compose ps
```

Consultar los mensajes:

```
docker compose logs
```

Seguirlos mientras se producen:

```
docker compose logs -f
```

Detener y retirar los contenedores:

```
docker compose down
```

Los volúmenes persistentes no se eliminan con este comando.

VI.7.0.11. Persistencia

La composición define dos volúmenes:

```
ollama
open-webui
```

El primero conserva los modelos y datos gestionados por Ollama.

El segundo conserva la configuración y los datos de Open WebUI.

PlantUML Server no necesita persistencia para la configuración inicial.

Esto permite destruir y recrear los contenedores sin perder los datos que deben sobrevivir a su ciclo de vida.

VI.7.0.12. Actualización

Las imágenes pueden actualizarse mediante:

```
docker compose pull
docker compose up -d
```

Durante el desarrollo pueden utilizarse las etiquetas indicadas en la composición.

Para una versión cerrada y reproducible de la infraestructura deberán fijarse versiones concretas de las imágenes utilizadas.

VI.7.0.13. Verificación

VI.7.0.13.1. PlantUML

```
curl http://localhost:20025
```

VI.7.0.13.2. Ollama

```
curl http://localhost:20434
```

VI.7.0.13.3. Open WebUI

Abrir en el navegador:

```
http://localhost:20300
```

VI.7.0.14. Resultado

Al finalizar esta etapa, la máquina dispone de una capa de servicios identificable y reproducible:

```
Máquina IASI

  puertos IASI 20000-20999

  Docker

      red iasi
          PlantUML Server
              20025 -> 8080

          Ollama
              20434 -> 11434
              volumen ollama
              modelos
                  Llama
                  Qwen
                  DeepSeek

          Open WebUI
              20300 -> 8080
              volumen open-webui
```

La convención distingue claramente tres niveles:

```
nombre del servicio   -> comunicación dentro de Docker
puerto interno        -> contrato del contenedor
puerto 20xxx          -> publicación IASI hacia el exterior
```

La definición pertenece a la infraestructura.

La ubicación física puede cambiar.

Las direcciones utilizadas por cada consumidor deben reflejar siempre la topología real del entorno.

VI.8. Sistemas inteligentes

IASI no plantea el trabajo asistido por inteligencia artificial como la utilización de un único asistente.

El entorno se prepara para trabajar con **varios sistemas inteligentes**, cada uno con capacidades, contexto y formas de interacción diferentes.

La primera configuración de referencia utiliza:

- ChatGPT, como asistente general de razonamiento, análisis, diseño y elaboración.
- GitHub Copilot, integrado directamente en Visual Studio Code y orientado al trabajo sobre el código y el proyecto.
- Ollama, de forma opcional, como *runtime* para modelos locales o para servicios de modelos disponibles dentro de la infraestructura.

La capa incorpora además herramientas que permiten estructurar el trabajo y acceder a estos sistemas:

- OpenSpec, para mantener especificaciones persistentes que puedan ser utilizadas por distintos asistentes de desarrollo.
- Open WebUI, como interfaz web para trabajar con Ollama y otros proveedores compatibles.

La idea no es acumular modelos. El objetivo es disponer de **varios sistemas que puedan participar en el mismo proceso de ingeniería**, apoyados por una infraestructura común.

VI.8.0.1. ChatGPT

ChatGPT se utiliza como asistente general dentro del proceso IASI.

Puede utilizarse directamente desde la web o mediante la aplicación oficial para Windows.

Referencia oficial:

OpenAI - ChatGPT para Windows

La aplicación para Windows puede instalarse desde Microsoft Store.

No existe una dependencia técnica entre ChatGPT y el entorno de desarrollo. Su papel es acompañar el trabajo de análisis, diseño, documentación, revisión y construcción del proyecto.

VI.8.0.2. GitHub Copilot

GitHub Copilot se integra en Visual Studio Code, instalado previamente en la capa de desarrollo.

Referencia oficial:

GitHub - GitHub Copilot en Visual Studio Code

Para utilizarlo:

1. Abrir Visual Studio Code.
2. Iniciar sesión con la cuenta de GitHub.

3. Configurar GitHub Copilot desde el propio entorno.

La configuración inicial de Copilot en Visual Studio Code instala automáticamente las extensiones necesarias.

GitHub Copilot aporta asistencia directamente sobre el proyecto abierto en el editor: código, archivos, diagnósticos y contexto de trabajo.

VI.8.0.3. OpenSpec

OpenSpec no es un sistema inteligente ni un modelo. Es una herramienta de desarrollo dirigido por especificaciones (*Spec-Driven Development*, SDD) que permite mantener especificaciones y cambios persistentes para que puedan ser utilizados por distintos asistentes de programación.

Referencia oficial:

OpenSpec - Installation

En IASI, OpenSpec se instala como una **herramienta global del entorno de desarrollo**. La inicialización y configuración de OpenSpec dentro de cada proyecto pertenece al uso de la metodología y no a la preparación de la infraestructura.

OpenSpec requiere Node.js 20.19.0 o superior.

Comprobar la versión instalada:

```
node --version
```

Instalar OpenSpec globalmente mediante `npm`:

```
npm install -g @fission-ai/openspec@latest --allow-scripts=@fission-ai/openspec
```

Verificar la instalación:

```
openspec --version
```

Si Node.js se gestiona mediante `nvm`, los paquetes globales se instalan dentro del entorno del usuario y no es necesario utilizar `sudo`.

La instalación de infraestructura termina en este punto. Comandos como `openspec init` actúan sobre un proyecto concreto y forman parte del flujo de trabajo de IASI.

VI.8.0.4. Trabajo con varias IA

IASI considera especialmente valioso que distintos sistemas inteligentes puedan trabajar sobre un mismo problema.

```
Ingeniero
  ChatGPT
      análisis, diseño, razonamiento, documentación

  GitHub Copilot
       código, proyecto, editor, implementación
```

No se trata de sustituir un sistema por otro.

Cada sistema observa el problema desde un contexto diferente. La interacción entre ellos permite contrastar decisiones, repartir tareas y utilizar la herramienta más adecuada en cada momento.

El ingeniero mantiene la dirección del trabajo, decide qué contexto recibe cada sistema y evalúa las propuestas obtenidas.

VI.8.0.5. Ollama

Cuando se necesite ejecutar modelos localmente, IASI Labs utiliza Ollama como *runtime* de modelos.

En esta infraestructura Ollama se ejecuta como servicio Docker y se publica en:

```
http://localhost:20434
```

La descarga de modelos no forma parte de la instalación base. Los modelos concretos dependerán del propósito del entorno y de los recursos disponibles.

VI.8.0.6. Open WebUI

Open WebUI proporciona la interfaz web para trabajar con Ollama y otros proveedores compatibles.

En esta infraestructura se ejecuta mediante la composición Docker definida en el capítulo anterior y queda disponible en:

```
http://localhost:20300
```

La comunicación entre Open WebUI y Ollama utiliza la red Docker `iasi-labs`; no requiere una segunda instalación de ninguno de los dos productos.

VI.8.0.7. Separación entre sistema y modelo

IASI distingue entre el **sistema inteligente** con el que se trabaja y el **modelo** que ese sistema pueda utilizar.

Por ejemplo, ChatGPT o GitHub Copilot pueden ofrecer distintos modelos sin dejar de ser sistemas de trabajo diferentes.

Esta distinción es importante porque IASI pretende trabajar con varias IA, no simplemente cambiar de modelo dentro de una misma herramienta.

VI.8.0.8. Resultado

La capa inicial queda organizada en dos grupos:

```
Sistemas inteligentes
   ChatGPT
   GitHub Copilot
   Ollama
       modelos locales, cuando sean necesarios

Herramientas de apoyo
   OpenSpec
      especificaciones persistentes y flujo de trabajo
   Open WebUI
       interfaz para Ollama y otros proveedores compatibles
```

ChatGPT y GitHub Copilot constituyen la configuración inicial de trabajo asistido por varias IA.

Ollama proporciona una vía adicional para incorporar modelos locales o servicios propios cuando el proyecto lo requiera. Open WebUI ofrece una interfaz común para trabajar con ellos, mientras que OpenSpec aporta la capa de especificación persistente que puede ser compartida por los distintos asistentes de desarrollo.

VI.9. IASI

Con la máquina base, las herramientas de desarrollo y los servicios preparados, instalamos **IASI**.

En este punto hablamos únicamente del ejecutable `iasi`. Los productos del ecosistema se instalarán después y no forman parte de este paso.

```
Infraestructura
↓
Desarrollo
↓
Docker y servicios
↓
IASI
↓
Productos IASI
```

VI.9.0.1. Instalación

El ejecutable `iasi` debe instalarse mediante su mecanismo de distribución, sin clonar los repositorios fuente utilizados para desarrollarlo.

El mecanismo definitivo de instalación se documentará aquí cuando quede estabilizado.

Una vez instalado, debe poder verificarse desde cualquier terminal:

```
iasi --version
```

VI.9.0.2. Reinstalación

La instalación de `iasi` debe quedar automatizada de forma que podamos reinstalar el ejecutable cuando queramos, sin reconstruir manualmente su entorno ni descargar su código fuente.

El comando definitivo de reinstalación se fijará junto con el mecanismo de distribución del ejecutable.

A partir de este punto asumimos que `iasi` está disponible en el `PATH` y puede utilizarse para instalar y mantener los productos IASI que necesiten los laboratorios.

VI.10. Productos IASI

Con el ejecutable `iasi` ya instalado, incorporamos los productos IASI necesarios para ejecutar los labs.

`iasi` es el punto de entrada para instalar, actualizar y reconstruir estos componentes sin necesidad de descargar sus repositorios fuente.

Esta es una diferencia fundamental respecto a `iasi-infra`.

En la máquina de desarrollo de IASI existen los repositorios fuente porque necesitamos modificarlos, probarlos y publicarlos.

En la máquina de labs **no descargamos los repositorios fuente de los productos IASI**.

```
NO
   git clone iasi.quarto
   git clone iasi-lua
   git clone ...

SÍ
   instalar iasi.quarto
   instalar extensiones de iasi-lua
   instalar los productos IASI requeridos
```

VI.10.0.1. iasi.quarto

`iasi.quarto` debe instalarse desde una distribución publicada del paquete, no mediante `remotes::install_local()` sobre un repositorio clonado.

El mecanismo definitivo de distribución se documentará aquí cuando quede estabilizado.

Después de la instalación debe poder verificarse desde R:

```
library(iasi.quarto)
packageVersion("iasi.quarto")
```

VI.10.0.2. iasi-lua

Las extensiones Quarto se instalarán mediante el mecanismo de distribución de Quarto, sin necesidad de mantener el repositorio `iasi-lua` en la máquina.

Cada proyecto materializará en `_extensions/` las extensiones que necesite.

La instalación debe poder comprobarse con:

```
quarto list extensions
```

VI.10.0.3. Repositorio de labs

`iasi-org/iasi-labs` es el repositorio canónico de los laboratorios.

El usuario puede trabajar directamente con una copia local para seguir el material o crear, cuando el lab lo requiera, su propio repositorio de trabajo en su cuenta de GitHub.

El repositorio canónico está protegido frente a escritura para los participantes.

VI.10.0.4. Regla

Una dependencia IASI utilizada por los labs debe poder instalarse como producto.

Si para utilizar un componente necesitamos clonar su repositorio fuente y conocer su estructura interna, ese componente todavía no está suficientemente empaquetado para este entorno.

VI.11. Postinstalación

La postinstalación conecta las piezas de la máquina y comprueba que pueden utilizarse conjuntamente.

La secuencia inicial es:

```
Windows preparado
↓
software base
↓
desarrollo
↓
Docker y servicios
↓
IASI
↓
productos IASI
↓
validación
↓
checkpoint 02-iasi-labs-ready
```

VI.11.0.1. Productos

Verificar `iasi.quarto`:

```
library(iasi.quarto)
packageVersion("iasi.quarto")
```

Verificar las extensiones Quarto desde un proyecto que las utilice:

```
quarto list extensions
```

VI.11.0.2. GitHub

La máquina debe disponer de Git y GitHub CLI correctamente configurados.

Verificar:

```
git --version
gh --version
gh auth status
```

VI.11.0.3. Servicios

Arrancar la composición de labs:

```
docker compose up -d
```

Verificar:

```
docker compose ps
```

VI.11.0.4. Resultado

La máquina deja de ser una instalación genérica de Windows y se convierte en el entorno de ejecución de IASI Labs:

```
IASI Labs
  herramientas generales
  herramientas de desarrollo
  servicios Docker
  ejecutable IASI
  productos IASI instalados
  repositorio de labs
```

Resultado del Lab 10

Cuando todas las validaciones sean correctas se crea:

```
02-iasi-labs-ready
```

A partir de este punto, los laboratorios posteriores pueden asumir esta infraestructura como base.

Licencias

El proyecto **iasi** contiene materiales de distinta naturaleza. Cada tipo de material se distribuye bajo una licencia específica.

Copyright © 2026 Javier G. Grandez.

Código fuente

El código fuente, los scripts, las automatizaciones, las utilidades y los ejemplos ejecutables se distribuyen bajo la **Licencia MIT**, salvo que se indique expresamente otra licencia.

Esto incluye, entre otros:

- scripts escritos en R;
- filtros, extensiones y utilidades;
- archivos de automatización;
- herramientas de construcción y publicación;
- fragmentos y ejemplos de código reutilizables.

La Licencia MIT permite utilizar, copiar, modificar, integrar, publicar, distribuir, sublicenciar y vender el código, siempre que se conserve el aviso de copyright y el texto de la licencia.

El texto completo está disponible en:

Licencia MIT

Identificador SPDX: `MIT`.

Contenido editorial

Los textos, capítulos, diagramas, tablas e imágenes originales de la obra **Ingeniería aumentada por sistemas inteligentes** se distribuyen bajo la licencia **Creative Commons Attribution 4.0 International**, salvo que se indique expresamente otra licencia.

Esto incluye, entre otros:

- los documentos y capítulos escritos en formato `.qmd`;
- el manifiesto y los principios;
- los diagramas, tablas e ilustraciones originales;
- las versiones HTML y PDF generadas a partir de la obra.

Esta licencia permite compartir y adaptar el contenido, incluso con fines comerciales, siempre que se reconozca adecuadamente la autoría, se proporcione una referencia a la licencia y se indiquen los cambios realizados.

El texto legal completo está disponible en:

Creative Commons Attribution 4.0 International

Identificador SPDX: `CC-BY-4.0`.

Atribución

Para reutilizar el contenido editorial, se recomienda utilizar una atribución semejante a la siguiente:

> Javier G. Grandez, *Ingeniería aumentada por sistemas inteligentes*, 2026. Licencia CC BY 4.0.

Cuando el contenido haya sido modificado, deberá indicarse de forma razonable que se han realizado cambios.

Materiales de terceros

Los materiales pertenecientes a terceros conservan sus respectivas licencias, derechos de autor y condiciones de uso.

Su inclusión o referencia en este proyecto no implica que estén cubiertos por la Licencia MIT ni por la licencia CC BY 4.0.

Cuando corresponda, la autoría, la procedencia y la licencia aplicable se indicarán junto al material o en su referencia bibliográfica.

Alcance

La Licencia MIT se aplica al **código**.

La licencia CC BY 4.0 se aplica al **contenido editorial original**.

Cuando un archivo incluya una indicación de licencia específica, esa indicación prevalecerá para dicho archivo.